

Go-reloaded

Présentation du projet | 2026-2027

ynov INFORMATIQUE

Au programme

Le fil conducteur de cette capsule

1. Le projet en bref
2. Comment fonctionne l'outil
3. Les règles : nombres et casse
4. Les règles : ponctuation et grammaire
5. Les pièges à anticiper
6. Sur quoi vous serez évalués
7. La méthode conseillée
8. Votre boîte à outils Go

ynov INFORMATIQUE

Le projet en bref

Un outil en ligne de commande qui corrige et met en forme un texte automatiquement

Go

langage imposé
tout le projet est écrit en Go

2

arguments
un fichier d'entrée, un fichier de sortie

0

package externe
uniquement la bibliothèque standard

Réutiliser vos fonctions

Les fonctions écrites dans vos exercices précédents deviennent les briques du projet.

Manipuler des fichiers

Lire un texte depuis un fichier et écrire le résultat dans un autre, via l'API fichiers de Go.

Manipuler texte et nombres

Découper, transformer, convertir : chaînes de caractères et bases numériques.

ynov INFORMATIQUE

Comment fonctionne l'outil ?

Un fichier en entrée, des règles appliquées, un fichier en sortie

1. Le fichier d'entrée

sample.txt contient le texte brut, avec ses balises (hex), (up)…

2. Votre programme

Il lit le fichier, applique chaque règle de modification dans l'ordre.

3. Le fichier de sortie

result.txt contient le texte corrigé, sans aucune balise.

```
$ cat sample.txt
Simply add 42 (hex) and 10 (bin) and you will see the result is 68.
$ go run . sample.txt result.txt
$ cat result.txt
Simply add 66 and 2 and you will see the result is 68.
```

ynov INFORMATIQUE

Règles : les conversions numériques

La balise remplace le mot qui la précède par sa valeur en base 10



`(hex)` **Hexadécimal → décimal**

Le mot précédent est un nombre en base 16. On le remplace par sa valeur décimale.

ENTRÉE

```
1E (hex) files were added
```

SORTIE

```
30 files were added
```



`(bin)` **Binaire → décimal**

Le mot précédent est un nombre en base 2. On le remplace par sa valeur décimale.

ENTRÉE

```
It has been 10 (bin) years
```

SORTIE

```
It has been 2 years
```

ynov INFORMATIQUE

Règles : la casse des mots

Mettre en majuscules, en minuscules ou capitaliser le mot précédent

(up) **Tout en majuscules**

Entrée `Ready, set, go (up) !`

Sortie **`Ready, set, GO!`**

(low) **Tout en minuscules**

Entrée `I should stop SHOUTING (low)`

Sortie **`I should stop shouting`**

(cap) **Majuscule initiale**

Entrée `Welcome to the Brooklyn bridge (cap)`

Sortie **`Welcome to the Brooklyn Bridge`**

(up, 2) **Sur les N mots précédents**

Entrée `This is so exciting (up, 2)`

Sortie **`This is SO EXCITING`**

ynov INFORMATIQUE

Règles : la ponctuation

Coller la ponctuation au bon mot, comme en typographie anglaise

Ponctuation simple

. , ! ? : ;

Collée au mot précédent, suivie d'un espace.

ENTRÉE
```
over there ,and then BAMM !!
```
SORTIE
```
over there, and then BAMM!!
```

Groupes

. . . !?

Le groupe reste entier et se colle au mot précédent.

ENTRÉE
```
I was thinking ... You were right
```
SORTIE
```
I was thinking... You were right
```

Apostrophes

' … '

Toujours par paire, collées au(x) mot(s) qu'elles entourent.

ENTRÉE
```
describe me: ' awesome '
```
SORTIE
```
describe me: 'awesome'
```

ynov INFORMATIQUE

Règles : « a » devient « an »

Une règle de grammaire anglaise à appliquer automatiquement

a → an

a e i o u h

si le mot suivant commence par…

ENTRÉE

```
There it was. A amazing rock!
```

SORTIE

```
There it was. An amazing rock!
```

ENTRÉE

```
bearing a untold story inside you.
```

SORTIE

```
bearing an untold story inside you.
```

Bon réflexe : vérifiez aussi « A » en majuscule et un mot suivant qui commence par une majuscule.

ynov INFORMATIQUE

Les pièges à anticiper

Les exemples du sujet ne couvrent pas tout : posez-vous ces questions avant de coder

Les balises disparaissent

Une fois appliquée, la est retirée : aucun espace double ne doit rester.

Balise + ponctuation

Dans « times (up) , it », balise et virgule se suivent : dans quel ordre les traiter ?

(cap, 6) en fin de texte

Le nombre peut être grand : comment compter les mots en arrière ?

Apostrophes à plusieurs mots

« ' I am the most… world ' » : seuls les mots aux extrémités bougent.

Retours à la ligne

strings.Fields supprime les sauts de ligne. Votre sortie les conserve-t-elle ?

Arguments invalides

Fichier absent, mauvais nombre d'arguments : que fait votre programme ?

ynov INFORMATIQUE

Sur quoi vous serez évalués

Quatre axes, du plus visible au plus profond

Le résultat est juste
Chaque règle produit exactement la sortie attendue, sur des fichiers que vous ne connaissez pas.

Exécution

Le code est propre
Bonnes pratiques Go : code formaté (gofmt), fonctions courtes, noms explicites, pas de duplication.

Relecture

Le code est testé
Des fichiers de test unitaires (_test.go) qui couvrent chaque règle et ses cas limites.

`go test`

Vous maîtrisez votre code
Vous savez expliquer chaque ligne, vos choix et modifier le code en direct.

Échange

Les exemples du sujet ne suffisent pas : un programme qui ne passe que ceux-là ne passera pas l'audit.

ynov INFORMATIQUE

La méthode conseillée

Avancer règle par règle, en testant à chaque étape

1

Lire / écrire

Lire le fichier d'entrée et le recopier tel quel dans la sortie.

2

Découper

Transformer le texte en liste de mots manipulables.

3

Une règle à la fois

Nombres, puis casse, puis a/an. Une fonction par règle.

4

Tester

Un test unitaire par règle, avec les exemples du sujet et les vôtres.

5

Ponctuation

La garder pour la fin : c'est la règle la plus délicate.

Commitez à chaque règle qui fonctionne.
Un historique Git propre vous sauve quand une nouvelle règle casse les précédentes.

ynov INFORMATIQUE

Votre boîte à outils Go

Tout est dans la bibliothèque standard : pas besoin d'installer quoi que ce soit

os
Accéder aux fichiers et aux arguments

- `os.Args`
- `os.ReadFile`
- `os.WriteFile`

strings
Découper et transformer le texte

- `strings.Fields`
- `strings.ToUpper / ToLower`
- `strings.Join`

strconv
Convertir entre texte et nombres

- `strconv.ParseInt(s, 16, 64)`
- `strconv.Atoi`
- `strconv.Itoa`

testing
Écrire et lancer vos tests

- `func TestXxx(t *testing.T)`
- `go test ./...`
- `fichiers *_test.go`

À vos questions !

